SALES AND CUSTOMER ANALYTICS SYSTEM

Purplle Online Beauty Shopping Database Management System

Introduction

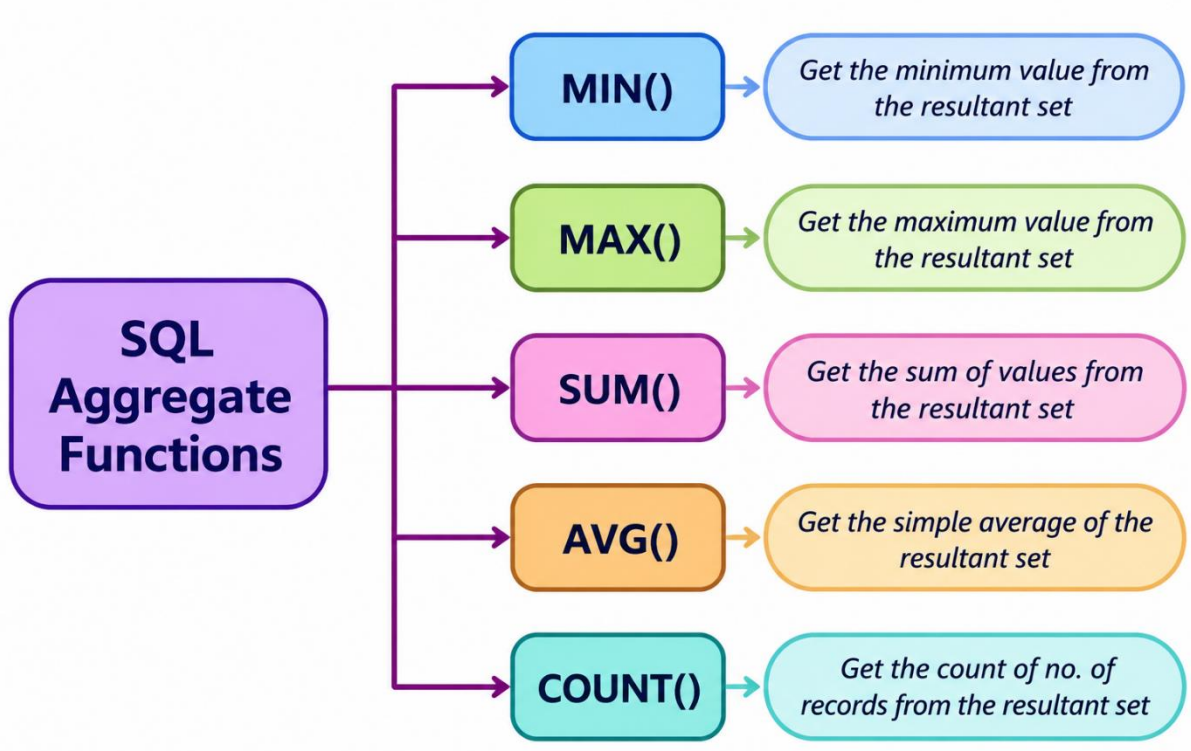
The Purplle Online Beauty Shopping Database Management System uses SQL aggregate functions to analyze sales and customer information. Aggregate functions such as COUNT(), SUM(), AVG(), MIN(), and MAX() are used to perform calculations on database records. These functions help identify total orders, sales amounts, and average purchase values. Customer-wise and product-wise reports are generated to understand shopping activities. This analysis helps manage business information efficiently.

Objectives

1. Apply COUNT(), SUM(), AVG(), MIN(), and MAX().
2. Generate total sales reports.
3. Find top customers based on purchase amount.
4. Identify best-selling products.
5. Perform category-wise sales analysis.

1. Aggregate Functions

Aggregate functions are SQL functions used to perform calculations on a group of rows and return a single result.



1.1 COUNT()

Find the total number of orders.

```
SQL> SELECT COUNT(Order_ID) AS Total_Orders
  2  FROM Orders;

TOTAL_ORDERS
------------
           5
```

1.2 SUM()

Find the total sales amount.

```
SQL> SELECT SUM(Total_Amount) AS Total_Sales
  2  FROM Orders;

TOTAL_SALES
-----------
       6145
```

1.3 AVG()

Find the average order amount.

```
SQL> SELECT AVG(Total_Amount) AS Average_Order_Amount
  2  FROM Orders;

AVERAGE_ORDER_AMOUNT
--------------------
                1229
```

1.4 MIN()

Find the lowest order amount.

```
SQL> SELECT MIN(Total_Amount) AS Minimum_Order
  2  FROM Orders;

MINIMUM_ORDER
-------------
          749
```

1.5 MAX()

Find the highest order amount.

```
SQL> SELECT MAX(Total_Amount) AS Maximum_Order
  2  FROM Orders;

MAXIMUM_ORDER
-------------
         1699
```

2. Generate Total Sales Reports

A sales report can be generated using the SUM() function on the Total_Amount column.

The output above is based on the sample Orders data used in your EcoCart project.

```
SQL> SELECT
  2      COUNT(Order_ID) AS Total_Orders,
  3      SUM(Total_Amount) AS Total_Sales,
  4      AVG(Total_Amount) AS Average_Order_Value,
  5      MIN(Total_Amount) AS Minimum_Order,
  6      MAX(Total_Amount) AS Maximum_Order
  7  FROM Orders;

TOTAL_ORDERS TOTAL_SALES AVERAGE_ORDER_VALUE MINIMUM_ORDER MAXIMUM_ORDER
------------ ----------- ------------------- ------------- -------------
           5        6145                1229           749          1699
```

2.1 Customer-wise Sales Report

Find how much each customer has purchased.

```
SQL> SELECT
  2      C.Customer_ID,
  3      C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  4      SUM(O.Total_Amount) AS Total_Purchase
  5  FROM Customer5 C
  6  JOIN Orders O
  7      ON C.Customer_ID = O.Customer_ID
  8  GROUP BY
  9      C.Customer_ID,
 10      C.First_Name,
 11      C.Last_Name
 12  ORDER BY Total_Purchase DESC;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_PURCHASE
--------------
        101
Nikitha R
          1699

        105
Sneha Patel
          1599

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_PURCHASE
--------------

        103
Arun Kumar
          1299

        104
Rahul Verma

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_PURCHASE
--------------
           799

        102
Priya Sharma
           749
```

Purpose: This report displays customer ID, customer name, total purchase amount, and customers arranged according to purchase amount.

3. Find Top Customers Based on Purchase Amount

Identify customers who have spent the highest amount.

```
SQL> SELECT
  2      C.Customer_ID,
  3      C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  4      SUM(O.Total_Amount) AS Total_Purchase
  5  FROM Customer5 C
  6  JOIN Orders O
  7      ON C.Customer_ID = O.Customer_ID
  8  GROUP BY
  9      C.Customer_ID,
 10      C.First_Name,
 11      C.Last_Name
 12  ORDER BY Total_Purchase DESC;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_PURCHASE
--------------
       101
Nikitha R
          1699

       105
Sneha Patel
          1599

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_PURCHASE
--------------

       103
Arun Kumar
          1299

       104
Rahul Verma

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_PURCHASE
--------------
           799

       102
Priya Sharma
           749
```

The customer appearing at the top has the highest purchase amount.

4. Identify Best-Selling Products

The Order_Details table contains the quantity of each product ordered.

Therefore, we can use SUM(Quantity) to find the total quantity sold for each product.

```
SQL> SELECT
  2      P.Product_ID,
  3      P.Product_Name,
  4      SUM(OD.Quantity) AS Total_Quantity_Sold
  5  FROM PurplleProduct P
  6  JOIN Order_Details OD
  7      ON P.Product_ID = OD.Product_ID
  8  GROUP BY
  9      P.Product_ID,
 10      P.Product_Name
 11  ORDER BY Total_Quantity_Sold DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
       101
Lakme Foundation
                  3

       103
Mamaearth Face Wash
                  2

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------

       105
Loreal Shampoo
                  2

       104
Minimalist Serum

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
                  1

       102
Maybelline Lipstick
                  1
```

Purpose:

1. Connects PurplleProduct and Order_Details.
2. Groups records according to product.
3. Calculates total quantity sold.
4. Displays products from highest to lowest quantity sold.

4.1 Product Sales Amount

Calculate the total sales generated by each product.

```
SQL> SELECT
  2      P.Product_ID,
  3      P.Product_Name,
  4      SUM(OD.Quantity) AS Total_Quantity_Sold,
  5      SUM(OD.Quantity * OD.Unit_Price) AS Total_Sales
  6  FROM PurplleProduct P
  7  JOIN Order_Details OD
  8      ON P.Product_ID = OD.Product_ID
  9  GROUP BY
 10      P.Product_ID,
 11      P.Product_Name
 12  ORDER BY Total_Sales DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_SALES
------------------- -----------
       101
Lakme Foundation
                  3        2397

       105
Loreal Shampoo
                  2        1599

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_SALES
------------------- -----------

       103
Mamaearth Face Wash
                  2        1299

       104
Minimalist Serum

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_SALES
------------------- -----------
                  1         799

       102
Maybelline Lipstick
                  1         749
```

This query displays both the quantity sold and sales amount for each product.

5. Category-wise Sales Analysis

Find the total quantity sold and sales amount for each product category.

```
SQL> SELECT
  2       C.Category_ID,
  3       C.Category_Name,
  4       SUM(OD.Quantity) AS Total_Quantity_Sold,
  5       SUM(OD.Quantity * OD.Unit_Price) AS Total_Sales
  6   FROM Category C
  7   JOIN PurplleProduct P
  8       ON C.Category_ID = P.Category_ID
  9   JOIN Order_Details OD
 10       ON P.Product_ID = OD.Product_ID
 11   GROUP BY
 12       C.Category_ID,
 13       C.Category_Name
 14   ORDER BY Total_Sales DESC;

CATEGORY_ID
-----------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_SALES
------------------- -----------
          1
Makeup
                  4        3146

          2
Skincare
                  3        2098

CATEGORY_ID
-----------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_SALES
------------------- -----------

          3
Haircare
                  2        1599
```

6. Combined Business Sales Report

Generate a complete report combining customer, order, product, and category information.

```
SQL> SELECT
  2       C.Customer_ID,
  3       C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  4       O.Order_ID,
  5       P.Product_Name,
  6       CAT.Category_Name,
  7       OD.Quantity,
  8       OD.Unit_Price,
  9       (OD.Quantity * OD.Unit_Price) AS Subtotal
 10   FROM Customer5 C
 11   JOIN Orders O
 12       ON C.Customer_ID = O.Customer_ID
 13   JOIN Order_Details OD
 14       ON O.Order_ID = OD.Order_ID
 15   JOIN PurplleProduct P
 16       ON OD.Product_ID = P.Product_ID
 17   JOIN Category CAT
 18       ON P.Category_ID = CAT.Category_ID
 19   ORDER BY O.Order_ID;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
  ORDER_ID
-----------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGORY_NAME
--------------------------------------------------------------------------------
  QUANTITY UNIT_PRICE   SUBTOTAL
---------- ---------- ----------
       101

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
  ORDER_ID
-----------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGORY_NAME
--------------------------------------------------------------------------------
  QUANTITY UNIT_PRICE   SUBTOTAL
---------- ---------- ----------
Nikitha R

CUSTOMER_ID
-----------
```

Report Structure

This report helps identify:

- Who purchased the product.
- Which order was placed.
- Which product was purchased.
- Which category the product belongs to.
- Quantity purchased.
- Amount generated.

Conclusion

The SQL aggregate functions were successfully implemented in the Purplle Online Beauty Shopping Database Management System. Different functions were used to calculate total, average, minimum, and maximum values. Customer-wise sales and product-wise reports were generated using SQL queries. These operations helped analyze sales performance and customer purchasing patterns. This task improved my understanding of SQL aggregate functions and database reporting.